Table of Contents

1. Design Considerations

1.1. Analysis Class Stereotype

1.1.1. Boundary Classes

Boundary classes are the interface between the system and its external environment, handling communication and data transfer between the system and users.

Classes such as Staff and Student implements the user interfaces for their respective user group while encapsulating external dependencies such as handling the reading and writing of camp information. These classes also interact with external files to store and retrieve camp-related data. This decision of encapsulation not only achieves a level of abrasion but also facilitates the testing of our classes' internal logic.

The AllCampToText class encapsulates external dependencies relating to file handling, which shields the system's internal logic from file handling.

Class Name	**Purpose**
Login	Verify and log users into their designated user group
StaffMenu	Manage their created camps
StudentMenu	Applying for camps, making enquiries and suggestions to available camps
CampCommitteeMenu	Assist in managing the camp and answering enquiries
AllCampToText	Import/Export camp information to an external text file

1.1.2. Control Classes

Control classes are responsible for coordinating and controlling the flow of data between the user interface, the entity classes (which represent the system's data), and any external systems.

We designed our Feedback system such that it consists of a controller and a handler class. The controller has the role of managing the incoming enquiries or suggestions made, whereas the handler class's role is to create incoming enquiries or suggestions. This feature ensures that each control class is responsible for its specific set of functions, which also adheres to the Single Responsibility Principle.

Class Name	**Purpose**
EnquiriesController	"Staff" and "Camp Committee" user group handles viewing and replying to enquiries
EnquiriesHandler	"Student" user group creates, edit, and delete enquiries
SuggestionController	"Staff" user group approves or rejects suggestions
suggestionHandler	"Camp Committee" user group creates, edit, and delete suggestions

1.1.3. Entity classes

Entity classes are usually persistent and encapsulate the attributes and behaviors of the entities within the system. Within our system, the entity classes are Camp; Staff; Student; CampCommittee; Enquiries; Suggestion; and User. These classes set their attributes as private to prevent direct access and provide controlled access through getters and setters.

1.2. Serializable interface

A Serializable interface signifies that an object of that entity class can be serialized. Serialization is a process of converting an object's state into a byte stream. We decided to implement a serializable interface so that we are able to store objects in a file and transmit them over the network.

We use classes like 'ObjectOutputStream' to write the objects of the class into a file and use 'ObjectInputStream' to read them back. This ensures object persistence such that the state of the object can be saved for later use. The usage of the Serialization interface also enhances cross-network communication. This way, we are able to send objects between different Java applications and even applications of a different language if they support serialization language. This displays loose coupling design principles.

1.3. Java Package

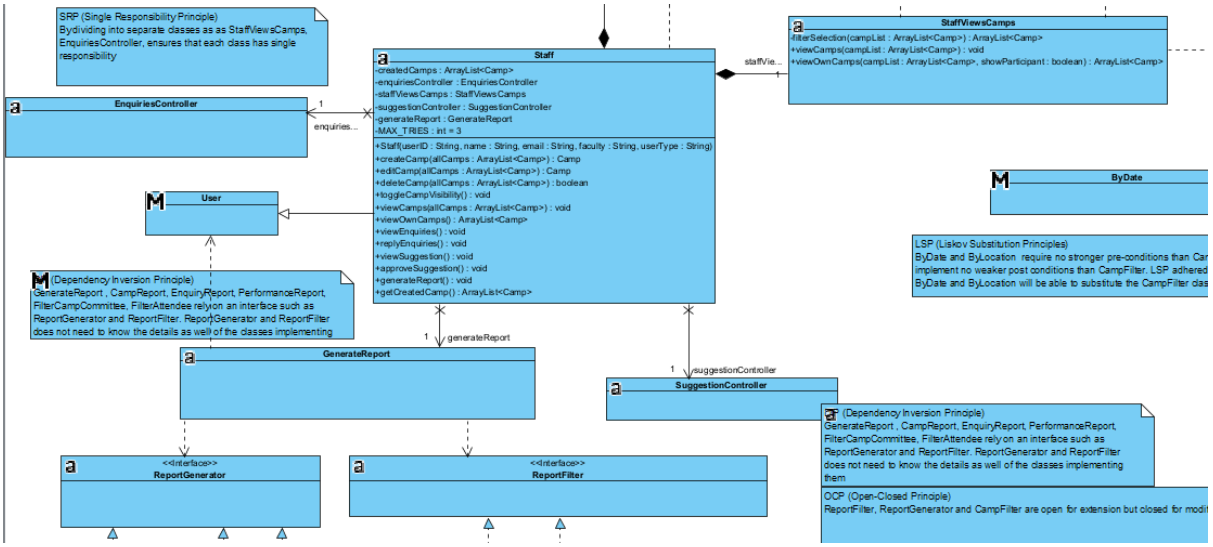
To ensure package cohesion, we organized the classes into packages based on the functionality they provide, such as Login, Staff, Student, Camp, CampCommittee, CampFilter, Feedback, and Report. Having a good package design ensures that each package represents a cohesive set of related classes that work together to implement a specific goal. Classes within the same package should be closely related in terms of functionality and work together as a cohesive unit to achieve a specific goal.

2. Design Principles Implemented

2.1. SOLID Design Principles

2.1.1. Single Responsibility

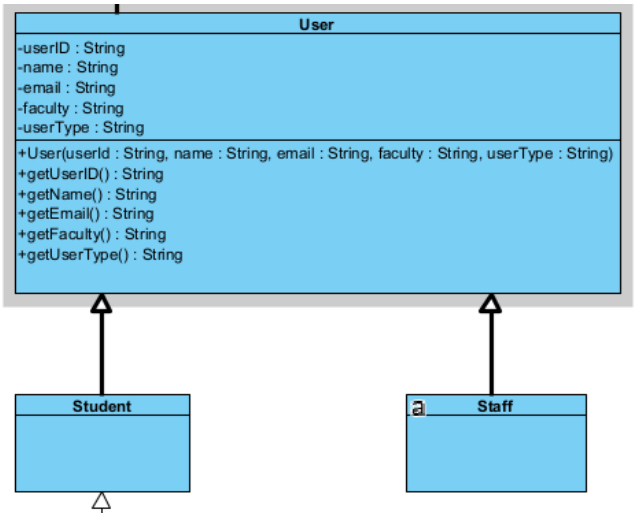
The Single Responsibility Principles state that each class should only have one responsibility. This means that a class will only have a singular, well-defined purpose or job and thus should only have one reason to change. By adhering to the SRP, it ensures that the class remains cohesive, where changes made will be restricted to its own domain.



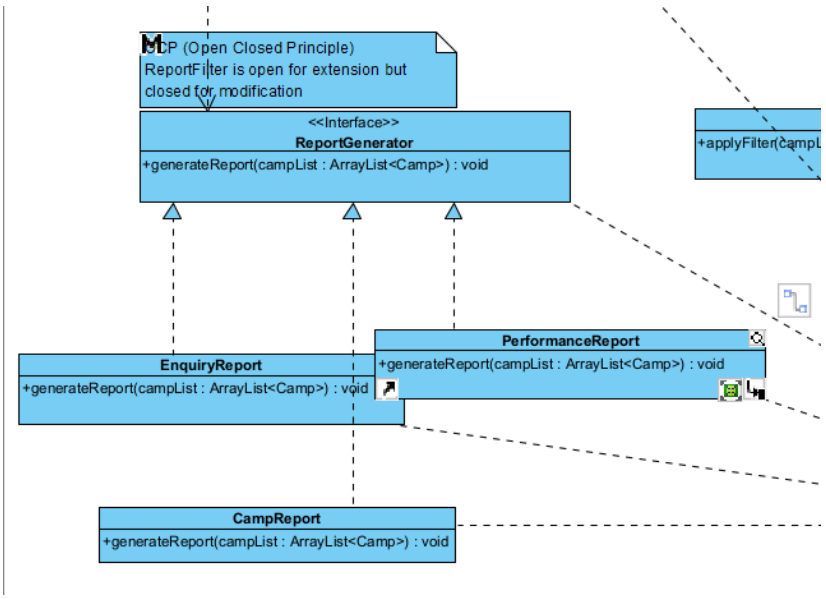
For example, by separating the classes into GenerateReport, EnquiriesController, StaffViewsCamps, and SuggestionController from Staff class, we ensure that each class has a single responsibility and that it fulfils this responsibility to the best of its extent.

2.1.2. Open/Closed

The Open/Close Principle states that the entities should be open for extension but closed for modification. This means that we should be able to add in new functionality to the class without making any modifications to the original source code.



An instance where OCP is incorporated is through the use of interfaces and abstraction. The User class acts as a base structure for defining the fundamental user attributes while the Student class and Staff class extend to it and integrate specialized functions to its respective user group. These extensions allow for future enhancements without making any modifications to the User class.



Another example is the ReportGenerator interface. The ReportGenerator is open for extension but closed for modification. This means that if new types of reports need to be added, the ReportGenerator need not be modified. The new types of reports can simply implement the ReportGenerator interface with their own unique generateReport functionality based on their report type.

2.1.3. Liskov Substitution

The Liskov Substitution Principle states that superclass objects should be replaceable with subclass objects without affecting the correctness of the program. This ensures that the subclasses are compatible with the base class and can be used interchangeably.

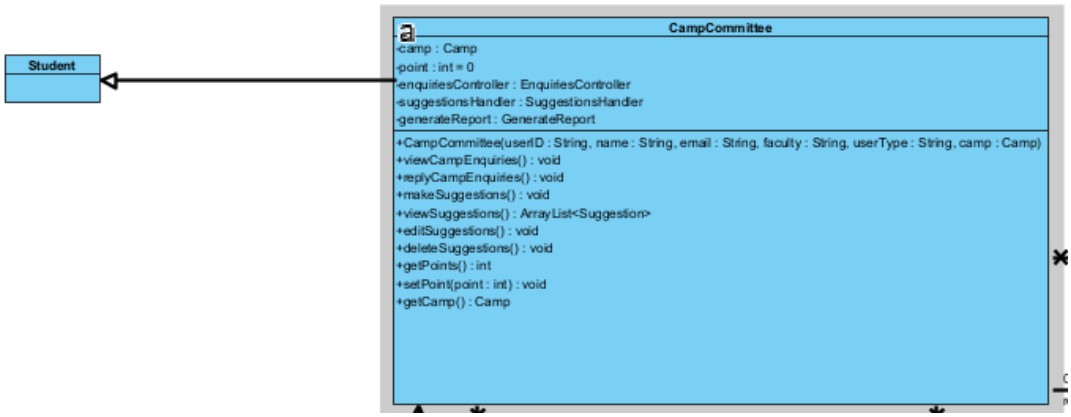
```
public abstract class CampFilter implements Serializable {

    public static ArrayList<Camp> sortByAlphabet(ArrayList<Camp> campList) {
        ArrayList<Camp> campListCopy = new ArrayList<>(campList);
        // Sort the copy using a custom comparator
        Collections.sort(campListCopy, Comparator.comparing(camp -> camp.getCampInfo().getCampName()));
        // Return the sorted copy
        return campListCopy;
    }
    public abstract ArrayList<Camp> applyCampFilter(ArrayList<Camp> campList);
}
```

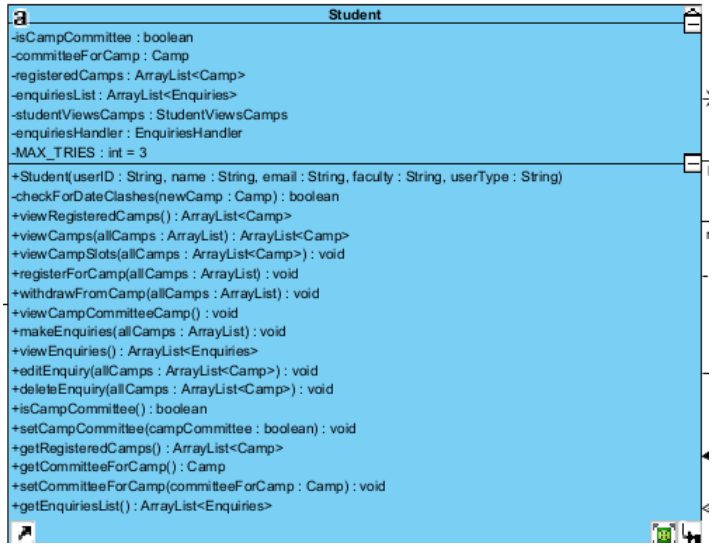
An instance where LSP was adhered to is the hieratical relationship between the CampFilter class and its subclass, ByLocation class and ByDate class. Polymorphism is shown through this hierarchical structure, which allows the code to treat subclass objects as instances of the same superclass.

CampFilter class:

Refer to Figures 1 and 2 for the ByLocation and ByDate class.

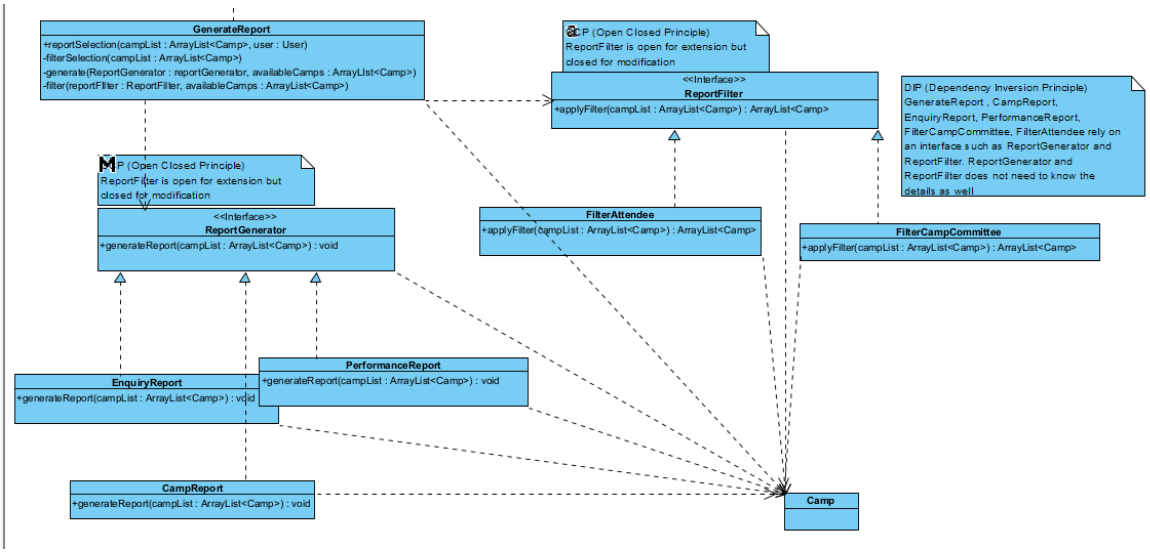
Liskov Substitution Principle is adhered here as both the ByData and ByLocation instances will be able to substitute the CampFilter class. This is because the preconditions and postconditions are satisfied since both the subclasses applyCampFilter correctly and due to inheritance, they can also sortByAlphabet.





LSP is also demonstrated here where the CampCommittee extends Student. Since every camp committee member is a student as well, he or she should be able to execute all the functionalities that a student is able to do. Hence, the pre-conditions are satisfied. Since CampCommitte members have additional features and power that a normal student doesn't have, the post-conditions are stronger than that of the student. As such Liskov's Substitution Principle is demonstrated here.

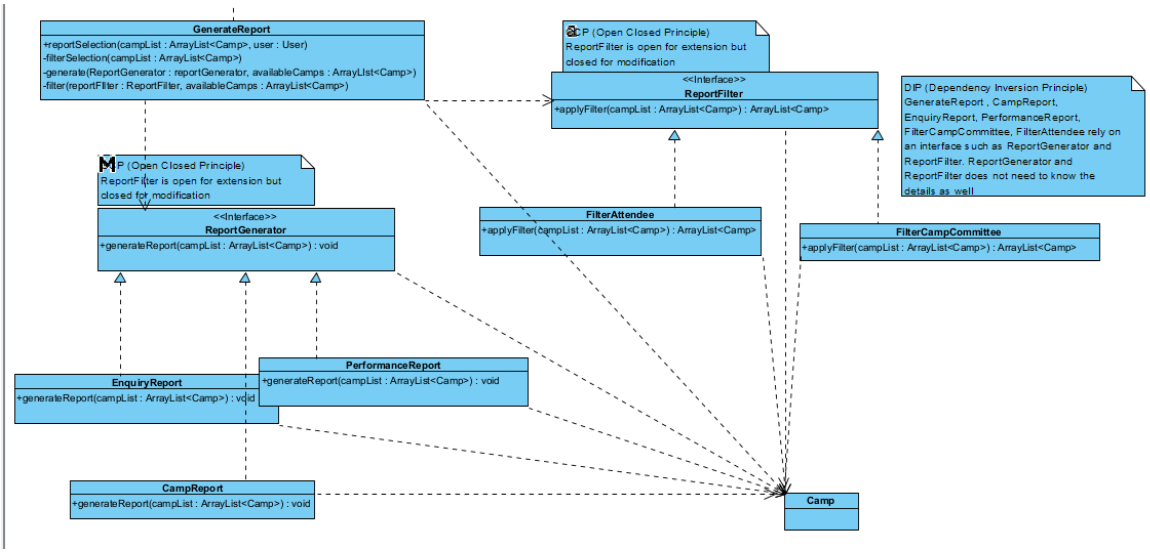
2.1.4. Interface Segregation



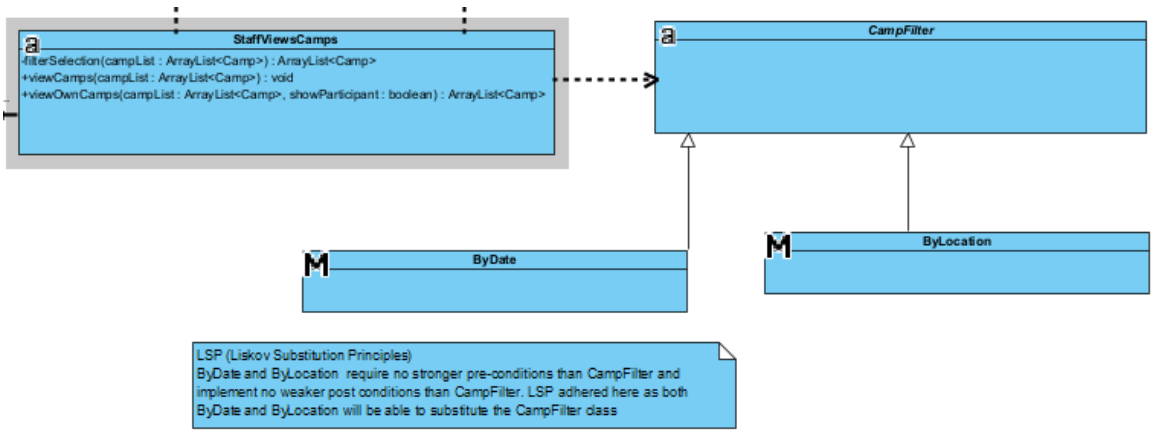
Interface Segregation is where a class should not be forced to implement interfaces it does not use. To ensure this, we segment our interface to more specific interfaces. (Refer to report package in UML diagram file). By separating into two specific interfaces, ReportGenerator and ReportFilter, we can prevent 'FAT' interfaces. It ensures that the classes are only dependent on the interfaces that they use. In this case, the purpose of the CampReport, EnquiryReport, and PerformanceReport is related to generating reports and hence only implement the ReportGenerator interface. FilterCampCommittee and FilterAttendee classes' purpose is to apply filters and hence only implement the ReportFilter interface.

2.1.5. Dependency Inversion

The Dependency Inversion Principle states that high-level modules should not depend on low-level modules, but both should depend on interfaces or abstract classes. To achieve dependency inversion, we implemented abstraction through Interfaces (or Abstract Classes), which allows for the definition of a common interface or set of methods that multiple concrete classes can implement.



Classes such as GenerateReport, CampReport, EnquiryReport, PerformanceReport, FilterCampCommittee, and FilterAttendee rely on the ReportGenerator and ReportFilter interfaces. The GenerateReport class, which utilizes the mentioned classes, will also depend on these two interfaces, allowing it to operate without detailed knowledge of how the mentioned classes work. As such, this satisfies the Dependency Inversion Principle (DIP).



Not only that, but also by creating an abstract class CampFilter, we were able to satisfy the Dependency Inversion Principle (DIP). The ByDate and ByLocation classes inherit from CampFilter, an abstract class. Consequently, the StaffViewCamps class only needs to be aware of the details of the abstract class CampFilter and can utilize the ByDate and ByLocation classes without needing to understand how they work.

3. Assumptions

- Only users with the right password can login into the system.
- The user is required to manually re-login after changing the password for the first time. (Run the program again.)
- The import and export of data occur before and after the completion of any user-initiated operation, respectively.
- After 3 unsuccessful attempts at user input, the user will be forced to return to the main menu.
- The student who is a camp committee of a camp will have an additional option in their student menu to access camp committee features.
- Students and camp committees cannot make enquiries or suggestions for a camp if its visibility is set to OFF.
- If a staff member doesn't approve a suggestion within 3 days, it is considered rejected.
- A camp committee knows everything about his/her camp.
- Pressing enter will register the input instead of moving on to the next line

4. Reflection

4.1. Difficulties Encountered

4.1.1. Getting started:

At the start of the project, we struggled to decide if we should start with the UML diagram first or go straight into coding. We decided to establish a base idea first and draw up a brief UML as starting guidelines for developing our code. We then proceeded to start coding.

4.1.2. Coding as a group

We initially assigned different tasks and parts of code to each other. However, there were often overlaps where two group mates were working on the same thing. To solve this issue, we decided to use GitHub to collaborate on our project. With GitHub, we were able to implement the necessary changes and resolve any conflicts in our codes smoothly. By using Git Bash, we were able to push and pull our code and remain in good sync with each other.

4.1.3. Designing the code with SOLID in mind:

Understanding that our code needed to be optimal, we had to think of a way to start coding with these principles in mind. It was crucial that we thought of it early before coding so that later on we would face lesser difficulties in tweaking our code to optimise it. We started off with the Single Responsibility Principle as it was one of the most crucial and straightforward principles to implement. The other principles slowly came along to ideation as we coded further.

4.1.4. Storing the dataset:

We faced difficulties when attempting to store our dataset and objects in Excel. To address this issue, we opted to use a text file instead.

4.1.5. Consolidating our UML diagram, report, and API:

Fine-tuning our UML diagrams requires good class diagram understanding. We needed to spend time learning how to use the visual paradigm. Including all the details also made it prone to make errors. Hence, it had to be done carefully and by solidifying our understanding. The process of developing the API and writing down their descriptions was also a time-consuming task.

4.2. Improvements To Be Made

4.2.1. User Input Handling:

A utility method could be created for reading and validating user input. This approach would help us encapsulate common user inputs into a single method instead of repeating the same user input handling input in multiple places.

For suggestions and enquiries, only one line of input is allowed, and pressing enter would register the input. Our code could be modified such that multiple lines of suggestions or enquiries (such as a paragraph can be written). However, we decided to standardize it to one line instead throughout the whole program.

5. UML Class Diagram

Refer to the UML Diagrams in the folder.